DAY 01

Full Stack Development Fundamentals

Easy-to-understand notes • Examples • Architecture diagram

Topic	What you will learn
Full Stack Development	How frontend, backend, and database work together.
Client–Server Architecture	How a browser/app communicates with a server.
HTTP and REST API	How requests and responses move information.
Python technology stack	Common tools for a Python web application.
Real-world example	How an online food ordering app works.

Goal: Understand the big picture first. You do not need to memorize every tool on Day 01.

1. What is Full Stack Development?

Full Stack Development means building both the part of an application users see and the part that works behind the scenes. It usually also includes a database for saving information.

A **Full Stack Developer** understands how these parts connect to make a complete application.

Example — online shopping: Product pictures and the Buy button are frontend. Checking prices and processing orders are backend. Saved product and order details are in the database.

2. The Three Main Parts

Part	Simple meaning	Example
Frontend	The screen and controls users interact with.	Product page, login form, buttons.
Backend	The logic that receives requests, checks data, and decides what to do.	Login check, order processing, permissions.
Database	Organized storage that keeps information for later.	User accounts, products, orders, marks.

Frontend technologies

HTML creates page structure. **CSS** controls appearance. **JavaScript** adds interactions. React and Angular are frameworks used to build larger interfaces.

Backend technologies

Python can be used for backend development. **Django**, **Flask**, and **FastAPI** help developers build web applications and APIs.

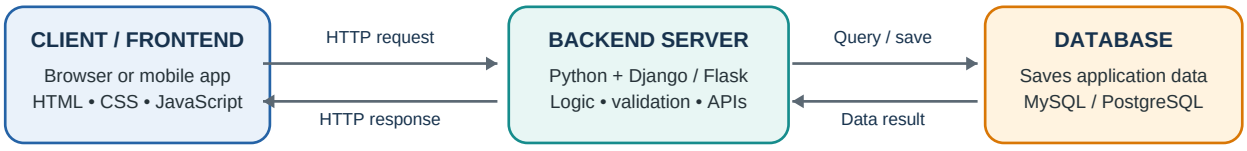
Database technologies

MySQL and **PostgreSQL** are relational databases. **SQLite** is also useful for learning and small projects.

Remember: Frontend = screen; Backend = logic; Database = saved information.

3. Client–Server Architecture

The **client** is usually a browser or mobile app. The **server** receives requests, runs backend code, may read or save database information, and sends a response.



User action → Frontend → Backend → Database (when needed)
Result returns to backend, then frontend displays it.

Safety rule: Frontend normally communicates with the backend, not directly with the database.

Read the diagram: (1) user clicks a button; (2) frontend sends a request; (3) backend processes it and may use the database; (4) backend sends a response; (5) frontend displays the result.

4. What is HTTP?

HTTP means Hypertext Transfer Protocol. It is a set of rules that allows a client and web server to communicate. When you open a website or submit a form, HTTP is commonly involved.

HTTPS is the secure version of HTTP. It encrypts information while it travels between client and server.

5. HTTP Request–Response Cycle

A **request** is a message the client sends to the server. A **response** is the message the server sends back.

Step	What happens	Example
1. User action	The user does something.	Clicks “Show Products”.
2. Request	Browser sends a request to an endpoint.	GET /api/products
3. Processing	Backend checks the request and may query the database.	Read product records.
4. Response	Server sends a status and result.	200 OK + product data
5. Display	Frontend uses the result.	Products appear on screen.

Common HTTP methods

Method	Meaning	Example
GET	Read or fetch information.	Get list of students.
POST	Send data, often to create something.	Register a student.
PUT	Replace or update a resource.	Update a full profile.
PATCH	Update part of a resource.	Change phone number.
DELETE	Remove a resource.	Delete cart item.

Common status codes

Code	Simple meaning
200 OK	The request worked.
201 Created	A new item was created.
400 Bad Request	The request or input is incorrect.
401 Unauthorized	Authentication is missing or failed.
403 Forbidden	The user is not allowed to do this.
404 Not Found	The requested item or endpoint was not found.
500 Internal Server Error	Something unexpected went wrong on the server.

6. What is an API?

API means Application Programming Interface. It defines how one software component can ask another component for information or an action.

Think of a restaurant waiter: you tell the waiter what you want, the waiter passes the request to the kitchen, and brings the result back. An API is like the interface between two systems.

7. What is a REST API?

A **REST API** is a common way for applications to communicate over HTTP. It organizes data as resources, gives resources URLs called **endpoints**, and uses HTTP methods to describe actions.

Request	Meaning
GET /api/students	Get the list of students.
GET /api/students/7	Get details for student number 7.
POST /api/students	Create a student record.
PATCH /api/students/7	Update part of student 7's record.
DELETE /api/students/7	Delete student 7's record, if permitted.

What is JSON?

JSON means JavaScript Object Notation. It is a text format often used to send structured data between frontend and backend.

```
{
  "id": 7,
  "name": "Asha",
  "course": "MCA"
}
```

The frontend asks for student details; the backend finds them and the API sends the result as JSON.

8. Python Full Stack Technology Stack

A **technology stack** is the group of tools used together to build an application. One possible beginner stack is:

Layer	Technology	Role
Frontend	HTML + CSS + JavaScript	Build the page and interactions.
UI framework (optional)	React or Angular	Organize a larger UI into reusable components.
Backend language	Python	Write application logic.
Python web framework	Flask, Django, or FastAPI	Handle requests and create APIs.
API communication	REST + JSON	Exchange information.
Database	MySQL or PostgreSQL	Store and retrieve information.
Tools	VS Code, Git, GitHub, Postman	Write code, track changes, test APIs.

Beginner tip: You do not need to learn all frameworks at once. Choose one Python framework, build a small project, then explore alternatives.

9. Real-World Example: Online Food Ordering

Imagine you use an app to order a pizza. Here is what happens behind the scenes:

Your action	Frontend does	Backend/database does
Open app	Shows restaurants and menus.	API retrieves restaurant and menu data.
Choose pizza	Shows selected item and cart.	Backend can check item and current price.
Place order	Sends order details to API.	Backend validates, calculates total, saves order.
See confirmation	Displays order number.	Backend returns order ID and result.
Check status	Shows order status.	Backend retrieves saved status or checks another service.

Important security idea

The backend must verify prices, permissions, and order information. It should not blindly trust data sent by the browser. The database should normally be accessed through the backend, not directly from the public frontend.

10. Full Stack Developer Workflow

Step	Work
1. Understand	Read the requirement and identify what the user needs.
2. Plan	Decide screens, API endpoints, data fields, and rules.
3. Build frontend	Create the page, forms, buttons, and layout.
4. Build backend	Write routes, logic, validation, and permissions.
5. Connect database	Design tables and save/retrieve information.
6. Integrate	Connect frontend actions to the API.
7. Test	Check normal cases, incorrect input, and errors.
8. Deploy and maintain	Publish the app, monitor it, fix bugs, improve it.

11. Quick Revision

- Full Stack Development combines frontend, backend, and database work.
- Client is usually the browser/app; server processes requests.
- A request goes to the server; a response comes back.
- HTTP methods include GET, POST, PUT, PATCH, DELETE.
- A REST API provides endpoints for applications to communicate.
- JSON is a common format for sending data.
- Python backend options include Django, Flask, and FastAPI.

12. Practice Task

Imagine you are creating a student registration website. Try answering:

1. What fields should the form contain?
2. What does the frontend show after registration?
3. What API endpoint could create a student?
4. What should the database store?
5. What should the server reply if the email is already registered?

13. Glossary

Word	Easy meaning
Client	Browser or app making a request.
Server	System that receives and processes requests.
Endpoint	URL path used to access an API feature.
Database	Organized storage for application data.
JSON	Text format for structured data.
CRUD	Create, Read, Update, Delete.
Validation	Checking that input is correct and allowed.
Framework	Tools and patterns that help build software.

End of Day 01 — Explain the architecture in your own words, then complete the practice task.